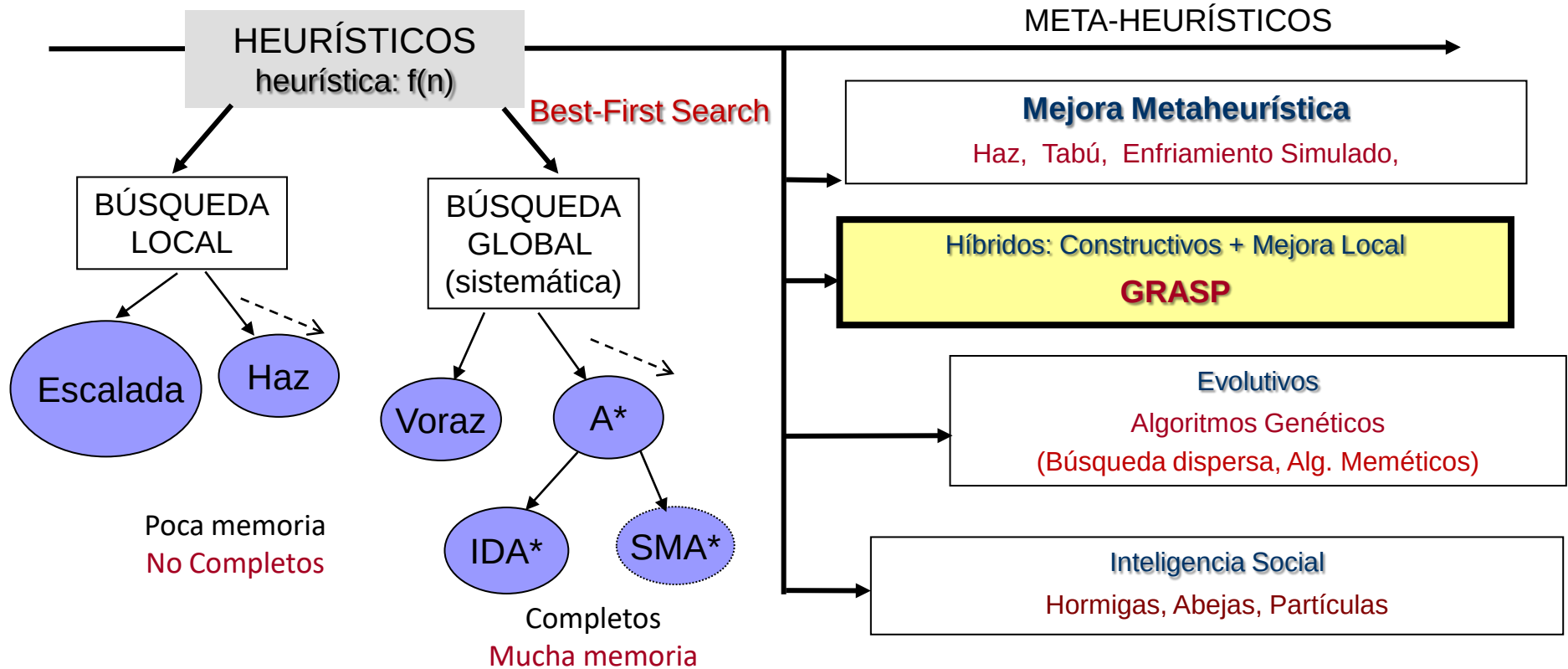
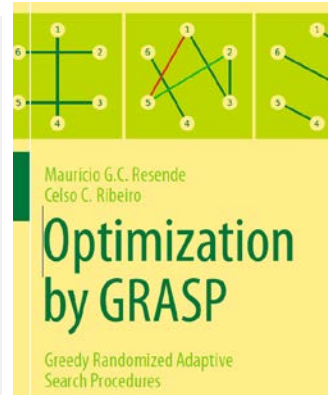


3.- Metaheurísticas Híbridas: Constructivos + Mejora Local



POLIFORMAT

- **GRASP: Greedy Randomized Adaptive Search Procedures.** M. Resende, J.L. González. Inteligencia Artificial, [Inteligencia Artificial](#). No.19
- **Optimization by GRASP.** M. Resende, C. Ribeiro. Springer (2016).
- **Machine Learning into Metaheuristics.** E.Talbi ACM Computing Surveys, 2021
- **Greedy Randomized Adaptive Search Procedures.** T.A. Feo, M. Resende. J. of Global Optimization 6 (1995)



Búsqueda Heurística en IA: Constructiva

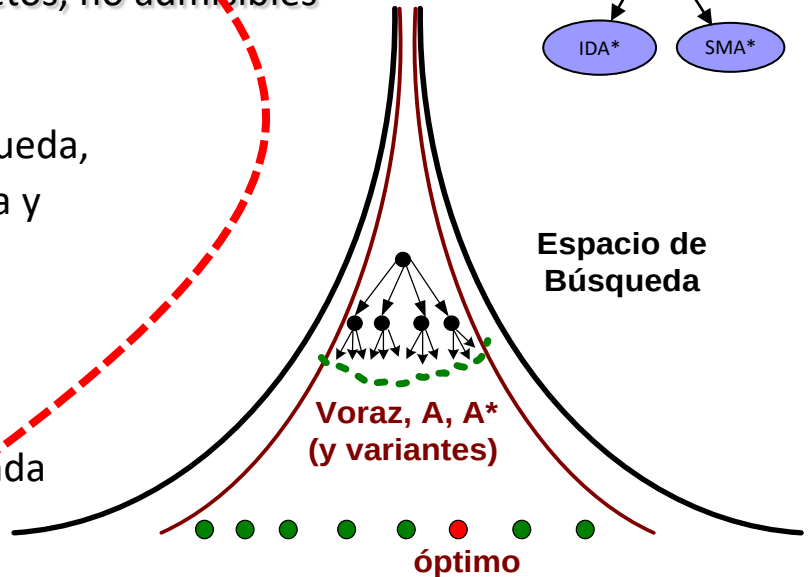
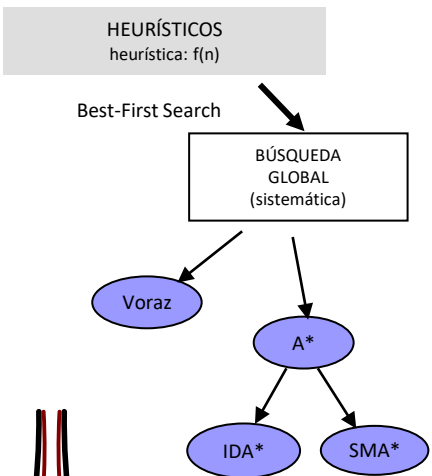
a) Búsqueda Global/Sistemática (típica)

Búsqueda Voraz (Greedy).

- Caso típico de “búsqueda el primero mejor”, donde no importa $g(n)$:
 $f(n) = h(n)$
- No siempre encuentran solución o acaban, no completos, no admisibles

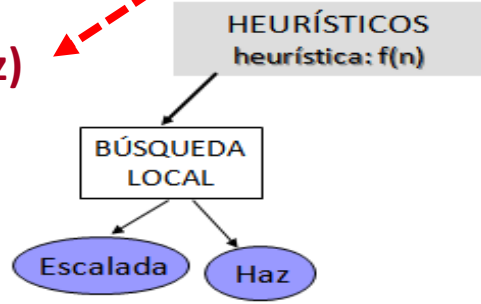
Algoritmo A.

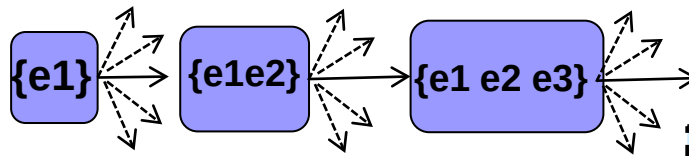
- Combinación de *búsqueda voraz* (reduce coste de búsqueda, pero no óptima ni completa) y *coste uniforme* (completa y óptima, pero ineficiente).
- $f(n)=g(n)+h(n)$
- Algoritmo A*: $\forall n, h(n) \leq h^*(n)$ Completa y admisible.
- Pero requiere mucha memoria (un algoritmo que expanda menos nodos que A* no tendrá admisibilidad).



b) Búsqueda Local (Escalada, Haz)

- Ciclos, Óptimos Locales, etc.





Construcción Solución Optimizada
(Pasos iterativos: +Elementos Solución)

Nuevos elementos se añaden iterativamente a la solución.

Métodos Constructivos:
Algoritmo A, A* $\Rightarrow$ **GRASP**

Mejora de una Solución
Requiere Solución Inicial
(Pasos iterativos: Mejora Solución)
Búsqueda en un espacio de Soluciones

Mejora Iterativa de **UNA SOLUCIÓN**

Enfriamiento Simulado
Búsqueda Tabú

Mejora Iterativa **CONJUNTO SOLUCIONES**
(no colaborativa)

Búsqueda en Haz

Mejora Conjunto Soluciones
(Pasos: Evoluciones Cooperativas)

Control Centralizado:
Evoluciones poblacionales
(Pobl. Individuos $\sim$ Soluciones)

Alg. Genéticos
Alg. Meméticos
Búsqueda Dispersa

Generación & mejora

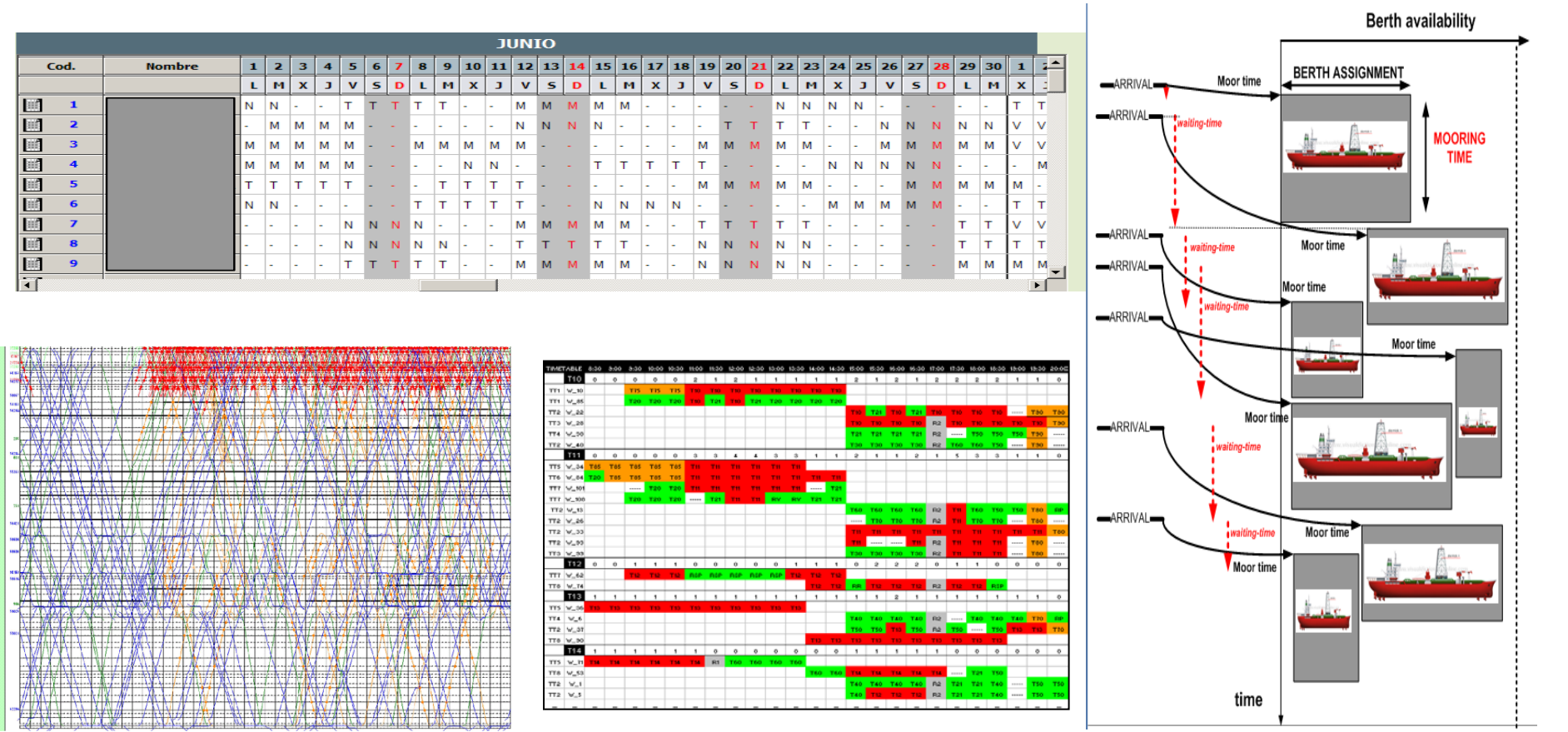
Control Autónomo:
Inteligencia de Enjambre
Enjambre de Individuos (Soluciones)

Alg. Hormigas
Colonia de Abejas,
Enjambre de Partículas...

Métodos Constructivos: Búsqueda de la solución en un espacio de estados: nuevos elementos se añaden iterativamente a la solución.

Apropiados para:

- Problemas en los que es difícil obtener soluciones iniciales (factibilidad)
- Vecindario muy amplio o poco estructurado (metaheurísticas de mejora)
- Complejidad de la representación de la solución (en metaheurísticas poblacionales)



“A probabilistic heuristic for a computationally difficult set covering problem”

Metaheurística de multi-inicio (**multi-start**) o iterativo, donde cada **iteración** tiene dos fases:

(i) Fase Constructiva: se encuentra una solución factible, mediante una heurística constructiva.

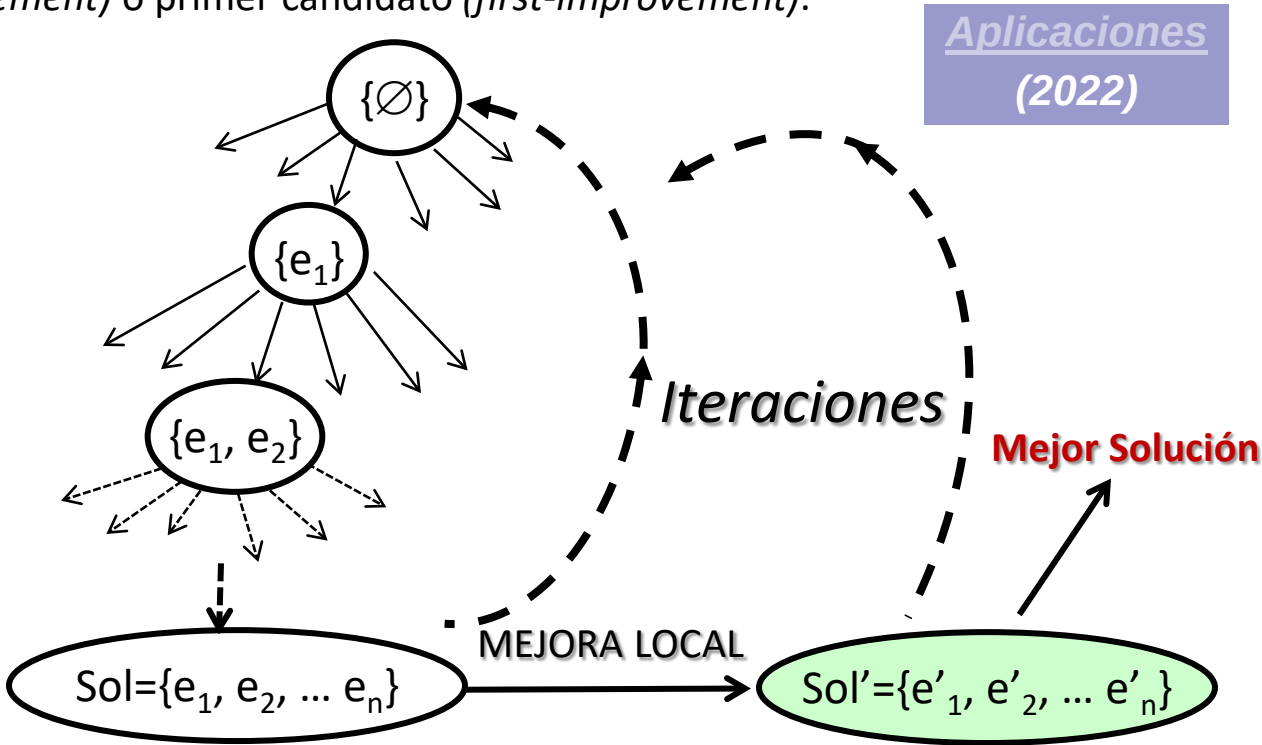
- No necesita una solución inicial,
- La longitud del camino (solución) debe ser limitada.
- Los elementos a añadir a la solución en cada paso son limitados (baja ramificación).

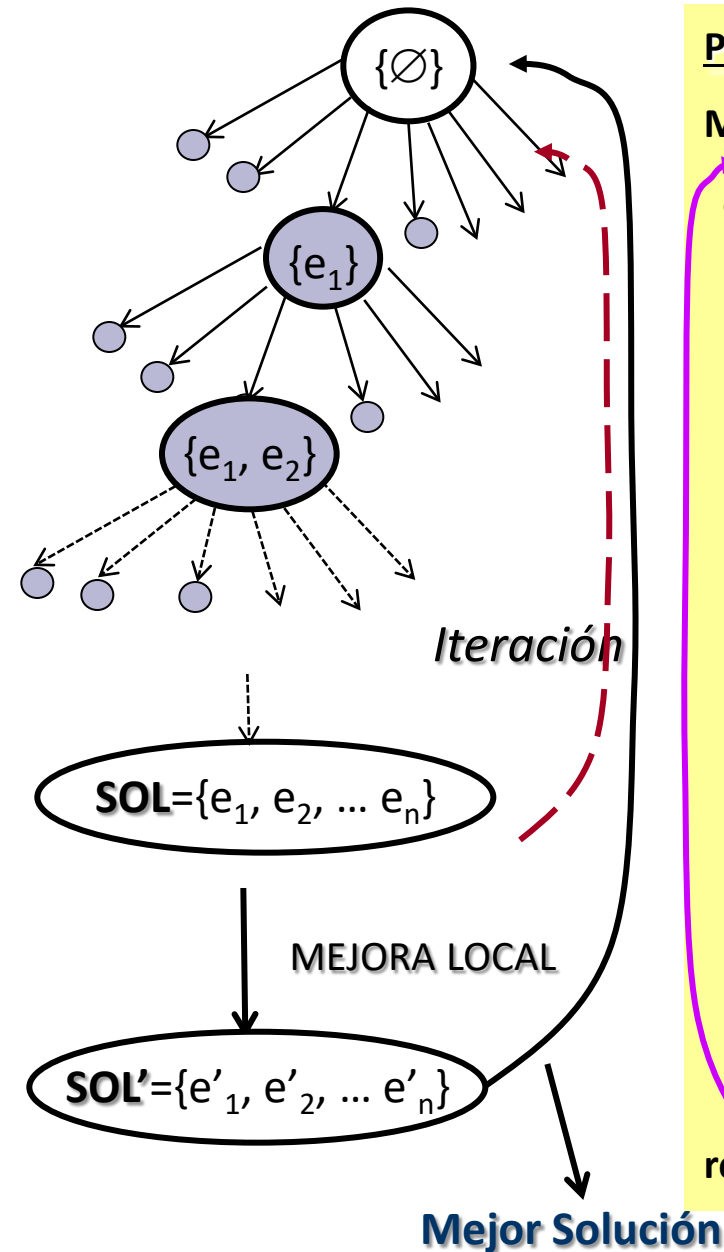
(ii) Fase de Mejora con Búsqueda local:

- La **vecindad** de la solución se explora para mejorarla, típicamente, mediante escalada: mejor candidato (*best-improvement*) o primer candidato (*first-improvement*).

**Al final de todas las iteraciones,
se guarda la mejor solución**

Aplicaciones típicas:
Viajante, Recubrimiento,
Scheduling, Diseño,
Path-finding,
Asignaciones, etc.





Procedimiento GRASP

$SOL_{MEJOR} = \emptyset, \alpha, f_{coste}$

Mientras (no condición de parada, o k iteraciones)

a) Fase Constructiva: $SOL = \emptyset$

Greedy Randomized Construction (Semilla)

Iterativamente:

- ➔ Formar lista de elementos candidatos "C". Evaluar ∇_{coste} .
 - ➔ Formar (α) lista restringida de los mejores candidatos (RCL).
 - ➔ Seleccionar un elemento aleatoriamente de la lista restringida.
- hasta encontrar solución (SOL).

b) Fase de Mejora: $SOL' = \text{busqueda local (SOL)}$

Proceso de búsqueda local a partir de la solución construida (SOL).

c) Actualización: $SOL_{MEJOR} = \text{Actualiza (SOL', SOL}_{MEJOR})$

Si la solución obtenida mejora a la mejor almacenada, actualizarla.

return (SOL_{MEJOR});

Fase Constructiva (greedy)

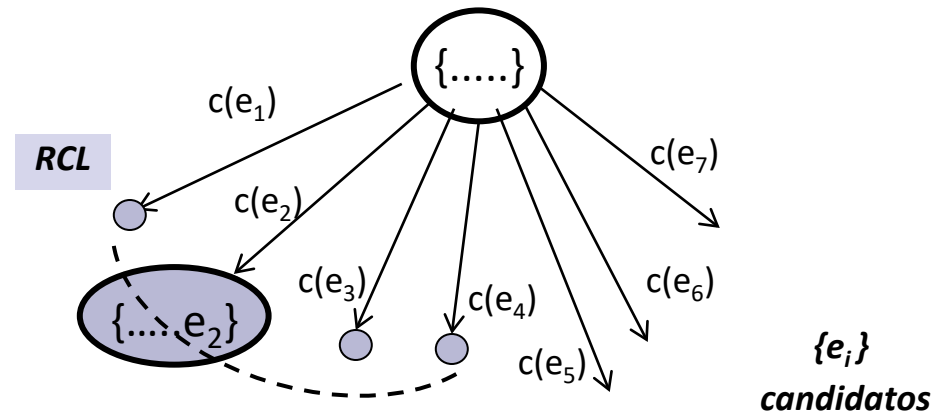
En cada iteración:

- **Elementos_Candidatos $C=\{e_i\}$:**

Conjunto de elementos factibles a incorporar a la solución parcial que se está construyendo.

La incorporación de cada elemento de $\{e_i\}$ tiene un **coste adicional $c(e_i)$** .

- **RCL:** lista de candidatos restringida (mejores elementos que incrementan menos, α , la función de coste).
- Elegir aleatoriamente un elemento de **RCL**.



La RCL esta formada por aquellos elementos e_i , tal que $c(e_i) \in \{c^{\min}, c^{\min} + \alpha(c^{\max} - c^{\min})\}$

α sirve para diversificar la construcción y no repetir soluciones:

$\alpha=0$ da un algoritmo completamente *greedy*, con $\alpha=1$ es una estrategia aleatoria.

- En cada iteración *no se selecciona al mejor candidato*, sino un elemento aleatorio del conjunto de mejores candidatos (RCL).

Fase Constructiva (greedy)

```
procedimiento Greedy Randomized Construction(Semilla)
 $s \leftarrow 0$ 
Inicializa el conjunto candidato:  $C \leftarrow E$ 
Evalua el costo incremental  $c(e) \forall e \in C$ 
while  $C \neq \emptyset$  do
     $c^{\min} \leftarrow \min\{c(e) | e \in C\}$ 
     $c^{\max} \leftarrow \max\{c(e) | e \in C\}$ 
     $RCL \leftarrow \{e \in C | c(e) \leq c^{\min} + \alpha(c^{\max} - c^{\min})\}$ 
    selecciona un elemento  $t$  de RCL aleatoriamente
     $s \leftarrow s \cup \{t\}$ 
    actualiza el conjunto candidato  $C$ 
    re-evalua los costos incrementales  $c(e) \forall e \in C$ 
return  $s$ 
```

- $C(e_i)$ representa el coste incremental de añadir e_i a la solución parcial.
- En cada iteración *no se selecciona al mejor candidato*, sino un elemento aleatorio del conjunto de mejores candidatos (RCL).
- En el caso que existan **restricciones de factibilidad**, $c(e) = \infty$ si $s \cup \{e_i\}$ no satisface restricciones.
- Coste incremental $c(e_i)$ puede ser obtenido como $h(e_i)$

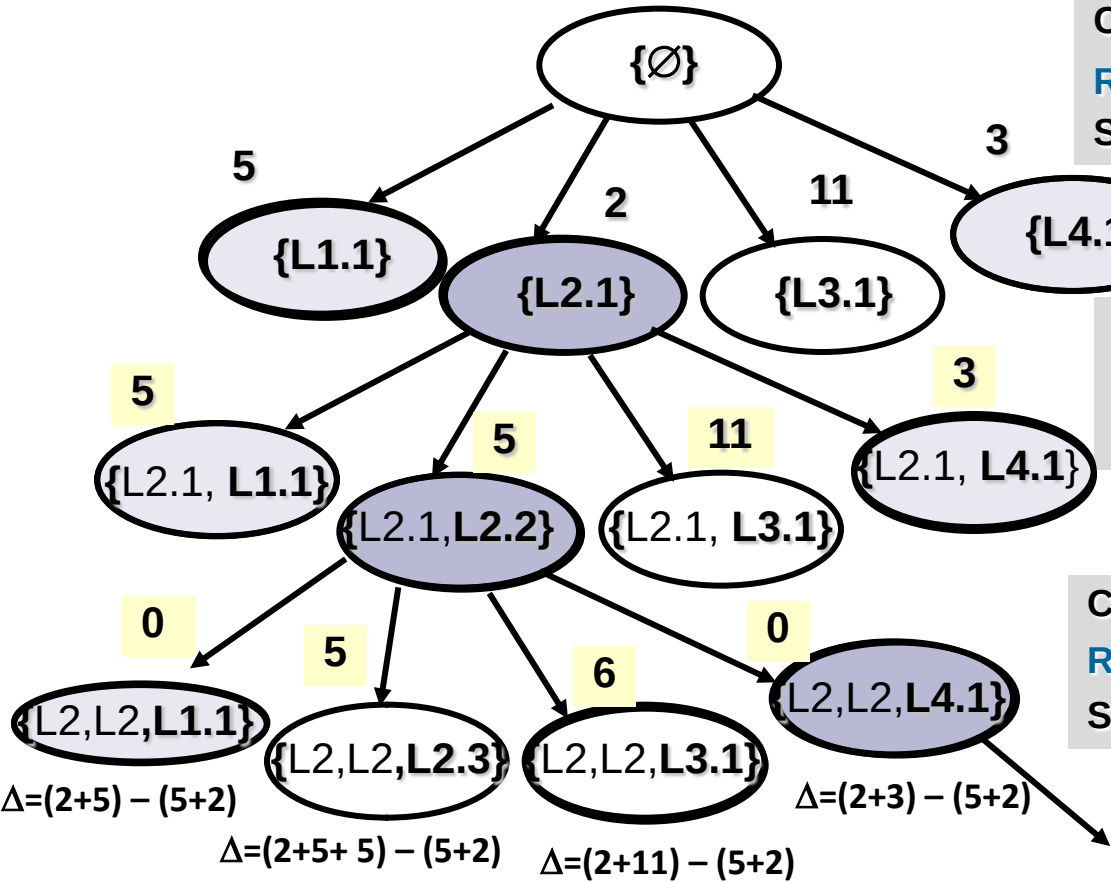
EJEMPLO-1

4 lectores (L1, L2, L3, L4) desean leer los periódicos (P1, P2, P3) en el mismo orden.

Greedy: Fase Constructiva

$S=\{\emptyset\}; \alpha=0,4$

	Ready-Time	P1	P2	P3
L1	0	5'	10'	2'
L2	0	2'	5'	5'
L3	0	11'	15'	15'
L4	0	3'	5'	5'

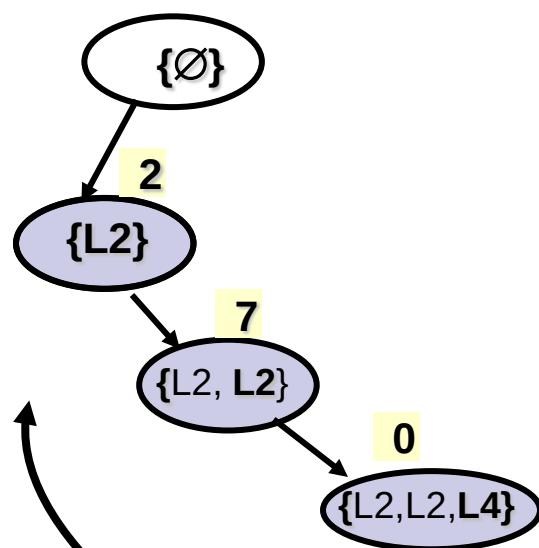


Costemin=2; Costemax=11;
 $RCL= \{L1, L2, L4\}; c(e) \leq 2 + 0,4 (11-2)= 5.6$
 $S = \{L2\};$

Cmin=3; Cmax=11;
 $RCL= \{L4, L1, L2\}; c(e) \leq 3 + 0,4 (11-3)=6.2$
 $S = \{L2, L2\};$

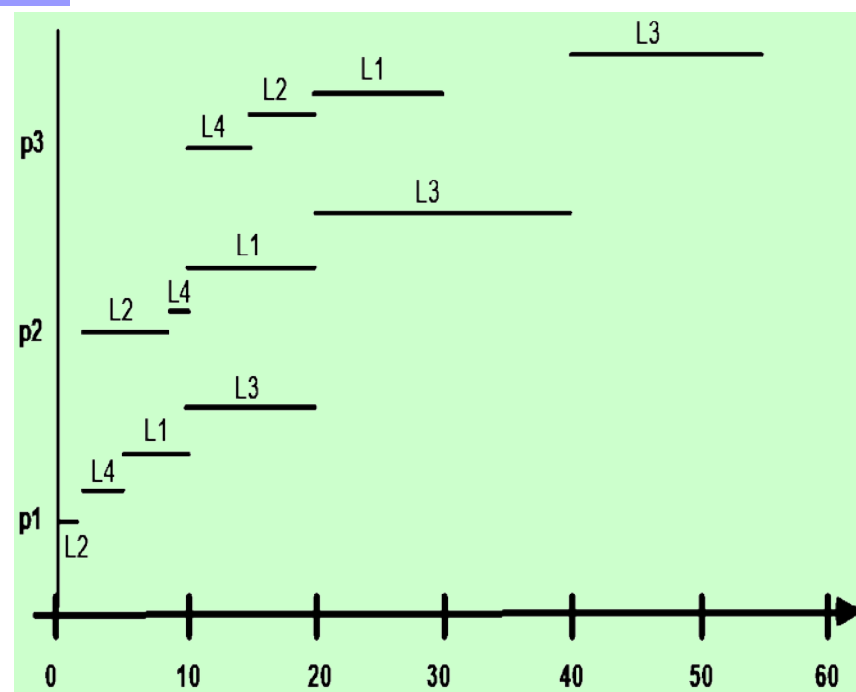
Cmin=0; Cmax=6;
 $RCL= \{L4, L3, L1\}; c(e) \leq 0 + 0,4 (6-0)=2.4$
 $S = \{L2, L2, L4\};$

	Ready-Time	P1	P2	P3
L1	0	5'	10'	2'
L2	0	2'	5'	5'
L3	0	11'	15'	15'
L4	0	3'	5'	5'



Iteración

SOL completa → SOL_Mejorada



GRASP: Importancia de la fase de Mejora Local

GRASP Semi-greedy: No tiene una fase de mejora local de la solución

En la fase constructiva, se obtienen soluciones optimizadas a partir de decisiones locales 'aleatorizadas' de la RCL

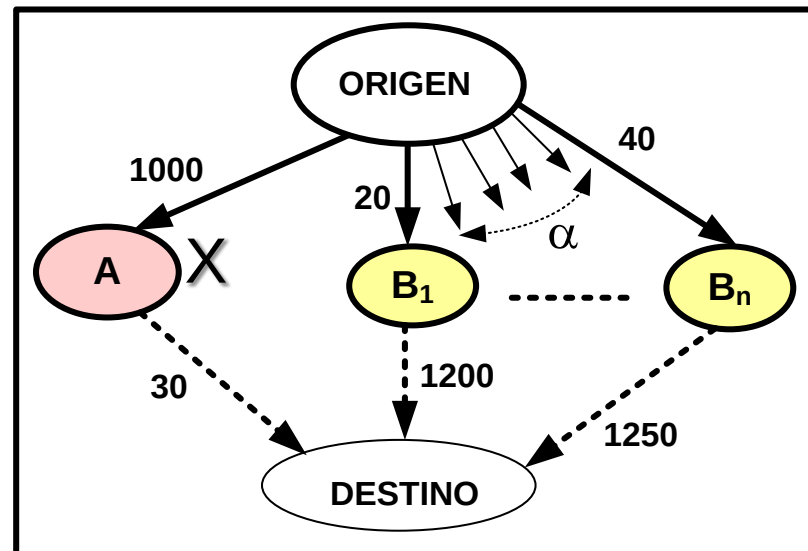
$$RCL \equiv \{c^{\min}, c^{\min} + \alpha(c^{\max} - c^{\min})\}$$

Como $\alpha < 1$, la peor (o peores decisiones) en un cierto nivel **van a ser siempre obviadas**

En el caso de que una 'pésima' decisión en un cierto nivel conduzca a la óptima solución, es imposible que GRASP pueda obtenerla, salvo en el proceso de mejora local final.

Cuando la solución óptima (o altamente optimizada) pueda estar compuesta de alguna decisión local donde la función de guía obtiene pésimas evaluaciones, un método constructivo puro no será adecuado.

GRASP tenderá a caer en máximos locales, en sucesivas iteraciones, sin que le sea posible alcanzar el máximo global



Características GRASP

- No requiere solución inicial. Aplicaciones: Problemas donde se requiere la ‘construcción’ de soluciones.
- Obtiene soluciones ‘optimizadas’. Mejora posterior por Búsqueda Local.

Variantes Metaheurística GRASP

Semi-greedy

- No tiene una fase de mejora local de la solución.
- Peligro de perder buenas soluciones si requiere un peor paso local.

Selección RCL

- Reactive GRASP:** Los ajustes de la RCL (α) se determinan dinámicamente según el estado de la búsqueda ($\approx$ enfriamiento simulado). Se inicia $\alpha \approx 1$ y va disminuyendo.
- Selección $e \in$ RCL no aleatoria:** Elitista, Ruleta, Rango, etc.

Clustering-GRASP, Path Relinking, etc.

- Se guardan las mejores soluciones (no solo la mejor) de los reinicios. Las metaheurísticas de mejora se inician desde diversas mejores soluciones previas
- No formar la RCL entre sucesores inmediatos del estado actual, sino los de un nivel siguiente, etc.

Aplicaciones

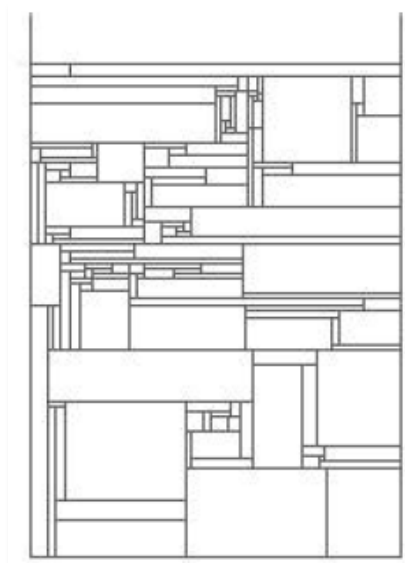
Apropiados para:

- Problemas en los que es difícil obtener soluciones iniciales (factibilidad)
- Vecindario muy amplio o poco estructurado (metaheurísticas de mejora)
- Complejidad de la representación de la solución (en metaheurísticas poblacionales)

JUNIO																																	
Cod.	Nombre	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	1	2
		L	M	X	J	V	S	D	L	M	X	J	V	S	D	L	M	X	J	V	S	D	L	M	X	J	V	S	D	L	M	X	J
	1	N	N	-	-	T	T	T	T	-	-	M	M	M	M	-	-	-	-	-	-	S	-	N	N	N	N	-	-	-	-	T	T
	2	-	M	M	M	M	-	-	-	-	-	-	N	N	N	-	-	-	-	-	T	T	T	T	-	-	N	N	N	N	V	V	
	3	M	M	M	M	M	-	-	M	M	M	M	M	-	-	-	-	-	M	M	M	M	M	-	-	M	M	M	M	V	V		
	4	M	M	M	M	M	-	-	-	N	N	-	-	-	-	T	T	T	T	T	-	-	-	-	N	N	N	N	N	-	-	-	M
	5	T	T	T	T	T	-	-	-	T	T	T	T	-	-	-	-	-	M	M	M	M	-	-	-	M	M	M	M	-	-	-	
	6	N	N	-	-	-	-	-	T	T	T	T	T	-	-	N	N	N	N	-	-	-	-	M	M	M	M	M	M	-	-	T	T
	7	-	-	-	-	N	N	N	N	-	-	M	M	M	M	M	M	-	T	T	T	T	T	-	-	-	-	-	-	T	T	V	V
	8	-	-	-	-	N	N	N	N	-	-	T	T	T	T	T	-	-	N	N	N	N	-	-	-	-	-	-	-	T	T	T	T
	9	-	-	-	-	T	T	T	T	-	-	M	M	M	M	M	-	-	N	N	N	N	-	-	-	-	-	-	-	M	M	M	M

JUNIO			
M	T	N	Tot
5	5	6	16
4	4	9	17
20	0	0	20
5	5	7	17
9	9	0	18
5	5	6	16
5	7	4	16
0	7	10	17
7	5	5	17

TIMETABLE		8:30	9:00	9:30	10:00	10:30	11:00	11:30	12:00	12:30	13:00	13:30	14:00	14:30	15:00	15:30	16:00	16:30	17:00	17:30	18:00	18:30	19:00	19:30	20:00
TT0		0	0	0	0	0	2	1	2	1	1	1	1	1	2	1	2	1	2	2	2	2	1	1	0
TT1	w_10				T15	T15	T15	T10	T10	T10	T10	T10	T10	T10											
TT1	w_05				T20	T20	T20	T10	T21	T20	T21	T20	T20	T20	T20										
TT2	w_22																								
TT3	w_28																								
TT4	w_30																								
TT2	w_40																								
TT4		0	0	0	0	0	3	3	4	4	3	3	1	1	2	1	1	2	1	5	3	3	1	1	0
TT5	w_24	T25	T25	T25	T25	T25	T11	T11	T11	T11	T11	T11	T11	T11											
TT6	w_04	T20	T25	T25	T25	T25	T11	T11	T11	T11	T11	T11	T11	T11											
TT7	w_01						T20	T20	T21	T11	T11	T11	T11	T11											
TT7	w_08						T20	T20	T20																
TT2	w_13																								
TT2	w_26																								
TT2	w_33																								
TT2	w_39																								
TT3	w_39																								
TT2		0	0	1	1	1	1	0	0	0	0	0	1	1	1	0	2	2	2	0	1	1	0	0	0
TT7	w_62						T12	T10	T10	RSP	RSP	RSP	RSP	RSP											
TT8	w_74																								
TT3		1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	2	1	1	1	1	1	1	0
TT5	w_36	T10	T10	T10	T10	T10	T10	T10	T10	T10	T10	T10	T10	T10											
TT4	w_6																								
TT2	w_37																								
TT8	w_90																								
TT4		1	1	1	1	1	1	1	0	0	0	0	0	0	1	1	1	1	1	1	0	0	0	0	0
TT5	w_71	T14	T14	T14	T14	T14	T14	R1	T60	T60	T60	T60													
TT6	w_53																								
TT2	w_1																								
TT2	w_5																								

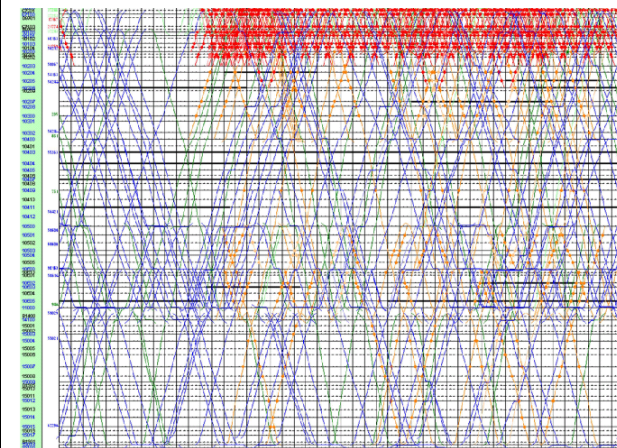
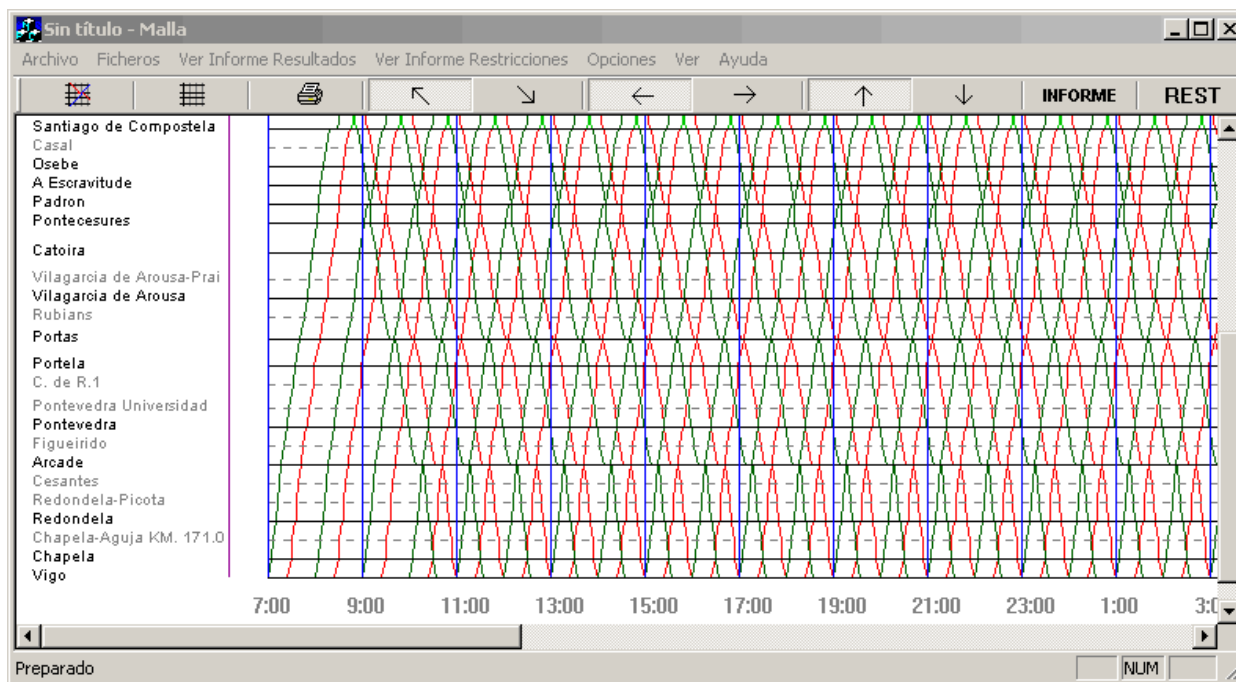


Representación gráfica de la secuencia de coste				Resumen	
Distribución de Coste				Resumen	Longitudinal (Cm.)
969637/1	969637/1	0.0	20567.7		
969655/1	969655/1	0.0	20382.4		
969655/1	969655/1	0.5	7989.2		
969655/1	969641/1	3.8	7545.5		
969655/1	969616/1	1.0	19001.6		
969642/2	969616/1	5.0	844.4		

Aplicaciones

Apropiados para:

- Problemas en los que es difícil obtener soluciones iniciales (factibilidad)
- Vecindario muy amplio o poco estructurado (metaheurísticas de mejora)
- Complejidad de la representación de la solución (en metaheurísticas poblacionales)



Aplicación Decisión GRASP en Planificación

Reshuffling a Container Yard with 6 rows x 6 stacks (maximum high: 4 containers):

To improve the planning performance,

- a domain-dependent heuristic function is based.
- the next action is randomly chosen from the actions that obtain a better performance of the heuristic function.

Algorithm 1: Pseudo-code of the domain-dependent heuristic function

Data: s : state to evaluate

Result: h , heuristic value of s

$h = 0$;

if $\exists x - \text{container} / \text{holding}(x) \in s$ then

 if $\text{goal-container}(x)$ then

$h = 0.1$;

 else

$h = 0.5$;

 end

end

for each row r in the yard-bay do

$\Delta h = 0$;

 for $x - \text{container} / \text{at}(x, r) \in s \wedge \text{goal-container}(x)$ do

 if $\nexists y - \text{container} / \text{goal-container}(y) \wedge \text{on}(y, x) \in s$ then

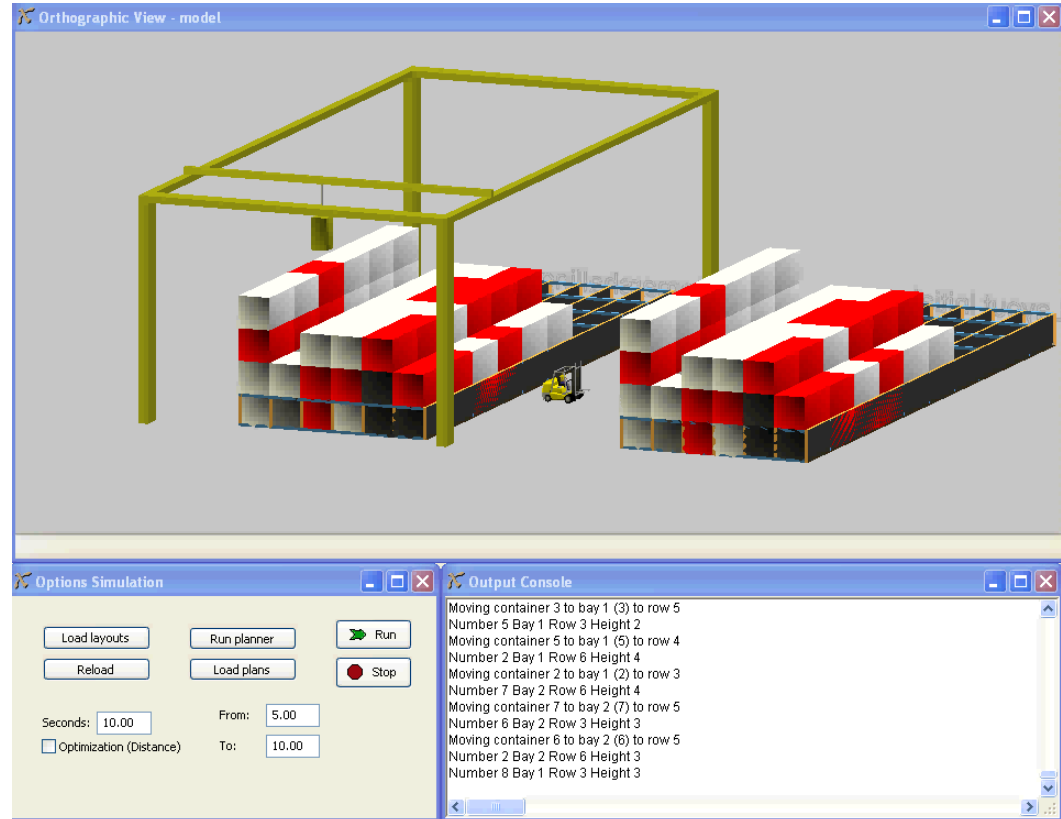
$\Delta h = \max(\Delta h, \text{numContainersOn}(x))$;

 end

 end

$h += \Delta h$;

end



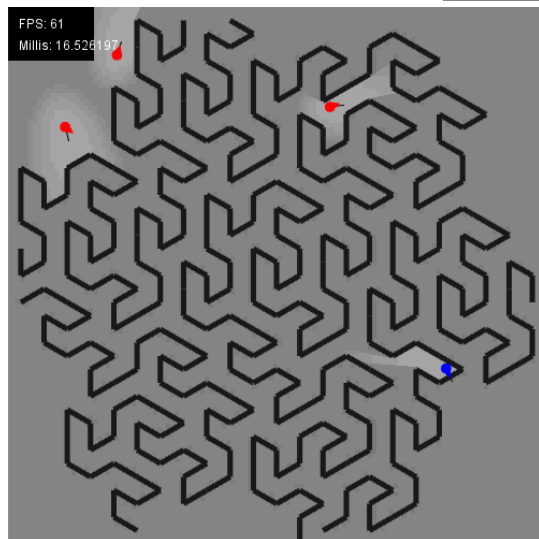
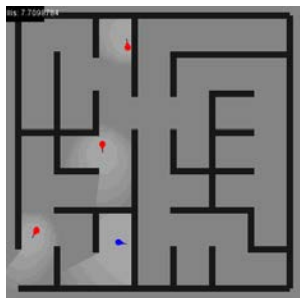
Path-finding: Problema de encontrar el mejor camino/ruta (habitualmente, el más corto) entre dos puntos a través de un mapa con obstáculos.

Problemas de laberinto, *Aplicaciones para videojuegos*, etc.

Algoritmos típicos: Dijkstra (muy costoso en laberintos complejos),
Algoritmos A (equivalente a Dijkstra, cuando $h(n)=0$),
A* sobre-informados, etc.

Metaheurísticas: Grasp, Inteligencia de Enjambre, Algoritmo de las hormigas, ...

objetivo



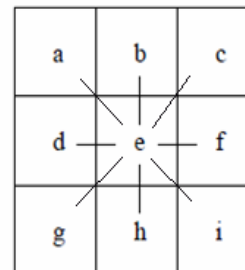
<http://www.movingai.com/benchmarks/>
Benchmarks for Grid-Based Path finding.
IEEE Trans. on Comp. Int. and AI in games

Grid-based path-finding: Descomposición del Mapa del terreno en Rejillas

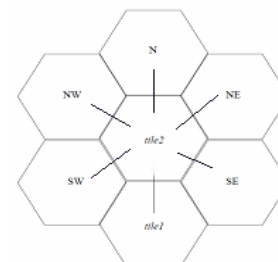


Rejilla de descomposición (Warcraft 3 Map)

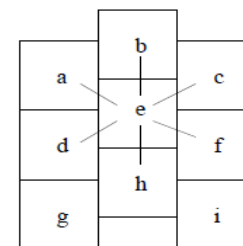
Alternativas de Descomposición:



Octiles

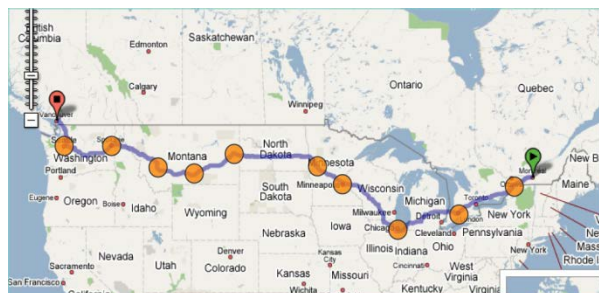
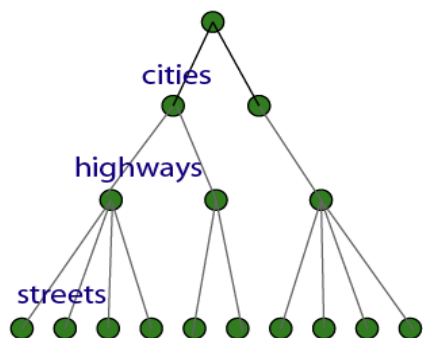


Hexes

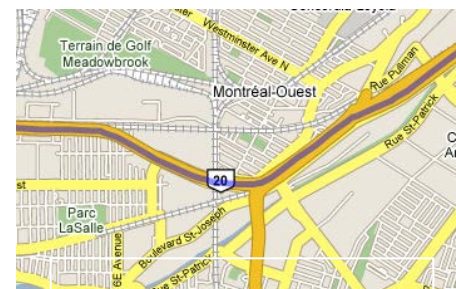


Texes

- A mayor nivel de descomposición, mayor nivel de ramificación.
- Técnica Jerárquica: **Path-finding jerárquico**



Paso 1) Búsqueda Conexiones Ciudades



Paso 2) Búsqueda de Calle

Path-finding: heurísticos básicos A, A*

Estimación distancia:

Manhatan, Euclidea, Hamiltoniana, etc.

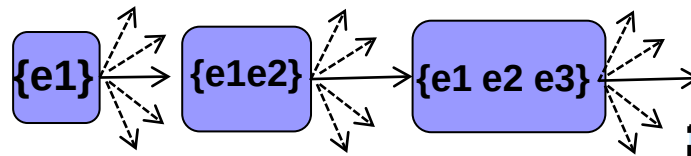
<https://qiao.github.io/PathFinding.js/visual/>

<http://ashblue.github.io/javascript-pathfinding/>

Path-finding con múltiples agentes

(deben mantener una cierta cohesión: PSO)

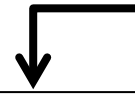




Construcción Solución Optimizada
(Pasos iterativos: +Elementos Solución)

Nuevos elementos se añaden iterativamente a la solución.

Métodos Constructivos:
Algoritmo A, A* $\Rightarrow$ GRASP



Mejora Iterativa de **UNA SOLUCIÓN**

Enfriamiento Simulado
Búsqueda Tabú

Mejora Iterativa **CONJUNTO SOLUCIONES**
(no colaborativa)

Búsqueda en Haz

Mejora de una Solución
Requiere Solución Inicial
(Pasos iterativos: Mejora Solución)
Búsqueda en un espacio de Soluciones

Mejora Conjunto Soluciones
(Pasos: Evoluciones Cooperativas)

Control Centralizado:
Evoluciones poblacionales
(Pobl. Individuos ~ Soluciones)

Alg. Genéticos
Alg. Meméticos
Búsqueda Dispersa

Generación & mejora

Control Autónomo:
Inteligencia de Enjambre
Enjambre de Individuos (Soluciones)

Alg. Hormigas
Colonia de Abejas,
Enjambre de Partículas...

